

Please write me lean4 code for solving the two state busy beaver problem.

Claude.ai

December 31, 2024

Contents

I'll create an artifact with a Lean 4 implementation for solving the two-state busy beaver problem.

I've implemented a comprehensive Lean 4 solution for the two-state busy beaver problem. Let me break down the key components:

1. Inductive Types:

- 'State': Represents the machine states (A, B, Halt)
- 'Cell': Represents tape cell values (Zero, One)
- 'WriteAction': Defines write operations (WriteZero, WriteOne)
- 'MoveDirection': Defines tape head movement (Left, Right)

2. Tape Representation:

- A 'Tape' structure that allows writing and moving the tape head
- Supports infinite tape by dynamically expanding left and right

3. Transition Rules:

- 'TransitionRule' defines how the machine behaves for each state and cell combination
- 'generateAllRules' creates all possible two-state transition rules

4. Machine Execution:

- 'transitionFunction' applies rules based on current state and cell

- ‘runMachine’ executes the Turing machine with a maximum step limit
- ‘findBusyBeaverChampion’ finds the machine that writes the most 1’s within the step limit

5. Halting Condition:

- Machine halts when it reaches the Halt state or no applicable rule is found
- Maximum steps prevent infinite loops

The ‘main’ function demonstrates how to use the implementation, finding the champion two-state busy beaver machine and printing its final state and steps taken.

A few notes on the implementation:

- It’s a theoretical exploration of the two-state busy beaver problem
- Uses Lean 4’s dependent type system for strong type safety
- Explores all possible two-state machine configurations
- Limits computation to prevent truly infinite loops

You can adjust ‘maxSteps’ to control the exploration depth. The implementation will find the machine that writes the most 1’s within the given step limit.

Would you like me to explain any part of the implementation in more detail?